

EduNexus

Advanced Learning Management System (LMS)

A PROJECT REPORT

Submitted By

Abhishek Rao (2023CA06)

Akanksha (2023CA10)

Santosh Kumar Mandal (2023CA84)

Under the guidance of

Dr. Indu Dohare

Assistant Professor, CSED Department

In Partial Fulfillment of the Requirements for the Award of the Degree of

Master of Computer Applications



Department of Computer Science and Engineering

Motilal Nehru National Institute of Technology Allahabad

Prayagraj, Uttar Pradesh - 211004

DECLARATION

We hereby declare that the project report entitled "**EduNexus**" submitted by us to the **MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD** during the academic year 2024–25 in partial fulfillment of the requirements for the award of Degree of Master of Computer Applications is a record of bonafide project work carried out by us under the guidance and supervision of **Dr. Indu Dohare**. We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other University.

Place : Prayagraj, Uttar Pradesh

Date : May 05, 2025

Abhishek Rao (2023CA06)

Akanksha (2023CA10)

Santosh Kumar Mandal (2023CA84)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Motilal Nehru National Institute of Technology Allahabad
Prayagraj, Uttar Pradesh - 211004



CERTIFICATE

This is to certify that the report entitled "**EduNexus**" submitted by **Abhishek Rao (2023CA06)**, **Akanksha (2023CA10)**, **Santosh Kumar Mandal (2023CA84)**, to the **MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD** in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications in Computer Science is a bonafide record of the project work carried out by them under my guidance and supervision. This report in any form has not been submitted to any other Universities or institutes for any purpose.

Place : Prayagraj, Uttar Pradesh

Date : May 05, 2025

Supervisor: **Dr. Indu Dohare**
Assistant Professor, CSED

ACKNOWLEDGMENT

We gratefully acknowledge the support and guidance of **Dr. Indu Dohare** , whose expertise and insights were invaluable throughout this project. We also thank the faculty and staff of the Department of Computer Science and Engineering for their encouragement. Finally, we appreciate the unstinting support of our families and friends.

Abhishek Rao (2023CA06)

Akanksha (2023CA10)

Santosh Kumar Mandal (2023CA84)

ABSTRACT

EduNexus is an advanced Learning Management System (LMS) developed using the MERN (MongoDB, Express, React, Node.js) stack. It is designed to cater to students and administrators by providing a seamless platform for course management, personalized career tools, and interactive learning features. The platform enables secure course enrollment and payment processing via Razorpay, along with integrated modules for quizzes, resume building, and AI-powered cover letter generation. These additions empower learners not only to acquire knowledge but also to prepare effectively for career opportunities. This report outlines the system requirements, detailed design specifications, implementation strategies, testing methodologies, and future enhancement plans. StepStudy aims to bridge the gap between conventional online education platforms and fully customizable, scalable web applications that support both academic learning and career readiness.

Contents

1	Introduction to the Project	8
1.1	Background and Context of the Project	8
1.2	Purpose of the Project	9
1.3	Project Objectives	9
2	System Analysis	11
2.1	Problem Analysis	11
2.2	Proposed System Solution	11
3	System Requirements	14
3.1	Hardware Requirements	14
3.2	Software Requirements	15
3.3	Functional Requirements	15
3.4	Non-Functional Requirements	16
4	System Design	18
4.1	Use Case Diagram	18
4.2	Class Diagram (Data Model)	20
4.3	Activity Diagram: Course Enrollment Process	21
5	Implementation	25
5.1	Authentication and User Management	25
5.2	Course Module Implementation	25
5.3	Quiz Generator	26
5.4	Resume Builder Implementation	26
5.5	Cover Letter Generator	27
5.6	Payment Integration	27
5.7	Other Features and Admin Tools	27
6	Testing	28
6.1	Test Plan and Methodology	28
6.2	Test Cases and Results	28

6.3	Summary	30
7	Results	31
7.1	Home	31
7.2	Login and SignUp Page	32
7.3	Managing User Profiles	32
7.4	Add to cart and Enrolled course	33
7.5	Payment	33
7.6	Quiz page	34
7.7	Quiz Result	34
7.8	Resume Builder	35
7.9	Cover Letter Generator	35
7.10	Available Categories of Course	36
7.11	Description of Course detail	36
7.12	Watching the video of the Course	37
7.13	Rating and Feedback	37
7.14	Adding Lecture By Instructor	38
7.15	ALL Course of Instructor	38
8	Conclusion and Future Work	39
8.1	Conclusion	39
8.2	Future Work	39

List of Figures

2.1	High-level architecture of the EduNexus platform, illustrating the MERN stack components and their interactions.	13
4.1	Use Case Diagram of the EduNexus system, depicting actors (Student, Instructor, Admin) and their interactions with the platform.	19
4.2	Class Diagram (ER Diagram) of the EduNexus system, showing the primary data models (User, Course, Quiz, Resume, etc.) and their relationships.	21
4.3	Activity Diagram illustrating the process of course enrollment and content access in EduNexus. It shows the student's journey from selecting a course, making payment, accessing lessons, to completing the course and giving feedback.	23
4.4	Sequence Diagram representing the flow of information.	24
7.1	Home Page	31
7.2	Login and signup	32
7.3	Managing user profiles.	32
7.4	Cart and enrolled course.	33
7.5	payment	33
7.6	quiz-page	34
7.7	Quiz Dashboard	34
7.8	Resume Builder	35
7.9	Cover Letter	35
7.10	Available-Categories	36
7.11	Course-detail	36
7.12	Watching the video	37
7.13	Rating and Feedback	37
7.14	New lec.. . . .	38
7.15	All course of instructor	38

Chapter 1

Introduction to the Project

1.1 Background and Context of the Project

Welcome to the world of **EduNexus**, a comprehensive web-based Educational Technology (EdTech) platform that modernizes the way learners interact with educational content and career development tools. In recent years, online learning has transformed education by making knowledge more accessible beyond the traditional classroom. However, this shift has also introduced challenges such as reduced interactivity and limited real-world skill application[8]. Many students struggle to translate what they learn into actionable career outcomes, and existing platforms often lack integrated tools to support both learning and job preparation.

The **EduNexus** system caters to a wide range of users—from beginners looking to acquire new skills to students preparing for real-world job opportunities. It offers a rich catalog of instructor-led courses across diverse domains (technology, science, arts, and more) delivered through structured video lessons and interactive assessments. What sets EduNexus apart is its combination of educational content with built-in tools for career readiness. These include an intuitive *resume builder*, an AI-powered *cover letter generator*, and personalized learning assessments via an AI-based *quiz module*.

Furthermore, EduNexus incorporates robust administrative features to maintain a high-quality learning ecosystem. The platform supports multiple user roles—*Student*, *Instructor*, and *Admin*—each with tailored functionalities. Instructors can seamlessly create and organize content into structured modules, while administrators oversee user management and content moderation. By combining scalable course management with career-oriented tools and secure infrastructure, EduNexus aims to empower learners with both knowledge and practical resources for professional success.

The rapid growth of the global EdTech and digital hiring tools market (projected to exceed \$250 billion by 2025[9]) underscores the importance of such an integrated platform in bridging the gap between education and employability.

1.2 Purpose of the Project

The primary purpose of the EduNexus project is to develop an all-in-one educational and career-enablement platform that combines academic learning with job preparation. In contrast to typical MOOCs or passive video lecture sites, EduNexus endeavors to create a *holistic learning environment* where course content, AI-generated quizzes, and professional tools (such as resume and cover letter builders) are integrated within a single system.

For students, EduNexus offers the convenience of self-paced learning augmented by engaging features such as dynamic quizzes, resume creation, and AI-assisted cover letter generation. These tools help ensure that learners are not just consuming content but are actively preparing for internships and job applications. For instructors, the platform provides intuitive tools to create structured courses and upload relevant multimedia content.

Another key goal of the project is to ensure a secure and user-friendly experience. This includes streamlined registration/login, role-based navigation, data protection, and integrated payment processing for course enrollments. EduNexus also supports user feedback mechanisms through ratings and reviews to improve course quality and transparency.

By fulfilling these purposes, EduNexus aims to enhance student learning outcomes, provide practical job preparation tools, and support the broader mission of making digital education impactful and career-relevant.

1.3 Project Objectives

The EduNexus platform is guided by several concrete objectives, defined to address the needs of modern learners and educators:

- **Comprehensive Course Delivery:** Enable instructors to create and organize rich course content (videos, sections) and publish it for learners across various domains and skill levels.
- **Interactive Learning Assessments:** Provide a quiz module where students can enter a topic and receive auto-generated quizzes via Gemini API, enabling self-evaluation and result tracking.
- **Career-Readiness Tools:** Integrate a resume builder that helps students format and generate professional resumes, and an AI-powered cover letter generator that creates customized letters based on job role, company, and user input.
- **Secure Transactions and Feedback:** Incorporate a trusted payment gateway for course purchases, along with a student-driven rating and review system to promote

quality assurance.

- **Robust User Management:** Support multiple roles (Students, Instructors, Admins) with role-based permissions, profile management, and administrative tools to ensure a secure and effective learning ecosystem.

In summary, the objective is to create a platform that not only delivers educational content but also fosters an interactive, career-aligned learning experience. This project was undertaken by a team of three members—*Abhishek Rao*, *Akanksha*, and *Santosh Kumar Mandal*—under the mentorship of *Dr. Indu Dohare*, Assistant Professor. Together, the team envisioned a system that embodies these objectives using modern web technologies.

The remainder of this report details how each objective is addressed: Chapter 2 presents the system analysis and rationale, Chapter 3 outlines the specific requirements, Chapter 4 discusses the system design with relevant diagrams, Chapter 5 describes the implementation of the features, Chapter 6 covers testing and quality assurance, and Chapter 7 concludes the report with insights and future directions. .

Chapter 2

System Analysis

2.1 Problem Analysis

The EduNexus project began with a detailed examination of common limitations in existing online learning platforms. Many traditional e-learning solutions rely heavily on static, pre-recorded video lectures with minimal interactivity. This one-way delivery model can lead to low engagement, reduced motivation, and a lack of personalized learning. Students often report feeling disconnected from the content and unsupported during their learning journey.

Furthermore, essential educational and career-preparation tools are often fragmented across different platforms. For example, one tool may be used to watch course videos, another to build a resume, and yet another to generate a cover letter. This fragmentation creates inefficiency, user frustration, and a lack of continuity in the overall learning experience.

From the instructor’s perspective, existing platforms may limit how content is structured, often lacking flexibility for modular course design or integrated assessment tools. Additionally, few platforms provide detailed analytics or progress-tracking features to help educators personalize instruction and improve learning outcomes.

Security and trust also remain pressing concerns. Learners need assurance that their personal information and financial data are protected. Instructors require reliable access controls to secure their content and ensure that earnings are accurately managed. Many smaller or custom-built e-learning systems lack robust user authentication, secure payment processing, or encryption protocols—undermining trust in the platform.

2.2 Proposed System Solution

In response to the challenges identified, the EduNexus platform was designed as a unified, interactive, and secure solution that blends course delivery with career-readiness tools.

The system addresses the core limitations of traditional e-learning environments through the following features:

- **Integrated Learning Ecosystem:** EduNexus consolidates course viewing, resume building, cover letter generation, and quiz-based self-assessment within a single web platform. This integration eliminates the need for users to juggle multiple applications, creating a streamlined and cohesive learning experience.
- **AI-Powered Quiz Generation:** To promote active learning, EduNexus allows students to generate quizzes based on a manually entered topic and question count using the Gemini API. This removes reliance on instructors to upload quizzes and fosters self-evaluation and personalized practice.
- **Career Enablement Tools:** The platform features a resume builder and an AI-based cover letter generator to support job applications. Students can format resumes and generate personalized cover letters tailored to specific roles and companies, improving their employability outcomes.
- **Scalable and Modern Architecture:** EduNexus is built using the MERN stack (MongoDB, Express.js, React.js, and Node.js), which provides performance, flexibility, and a unified development environment. The system is well-suited for cloud deployment and can scale horizontally to meet increasing user demand.
- **Secure Authentication and Transactions:** EduNexus uses hashed passwords and JSON Web Tokens (JWTs) for secure user authentication. All course payments are processed via Razorpay, a trusted payment gateway. Role-based access control and input validation further enhance platform security.
- **Feedback-Oriented Improvement:** Students can rate and review courses upon completion, offering insights to future learners and helping instructors refine their material. This continuous feedback loop ensures the platform remains responsive to user needs and expectations.

By integrating essential learning and career tools into a unified system, EduNexus addresses both academic and professional development needs. It empowers students not only to acquire knowledge, but also to apply it confidently in real-world scenarios.

In summary, the EduNexus system was developed by aligning user needs—engagement, integration, and security—with a robust technology stack. As shown in Figure 2.1, the React-based front-end communicates with the Node.js/Express back-end through secure HTTP API calls. The back-end handles logic for tasks such as user authentication, course retrieval, quiz generation, and payment verification. It also interfaces with MongoDB to

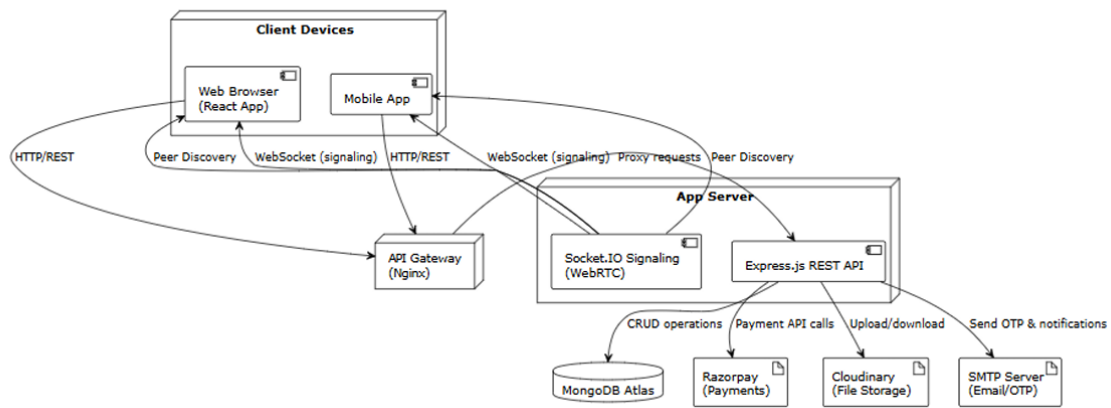


Figure 2.1: High-level architecture of the EduNexus platform, illustrating the MERN stack components and their interactions.

manage persistent data including user accounts, course content, resumes, quiz results, and transactions.

This architecture ensures the system remains responsive and intuitive for users while maintaining reliability, scalability, and security on the server side. It lays the foundation for

Chapter 3

System Requirements

A successful implementation of EduNexus requires a clear definition of system requirements, which encompass the environment needed to develop and run the application as well as the features the system must deliver. This chapter enumerates the hardware and software setup for the project and details the functional and non-functional requirements identified during the planning phase.

3.1 Hardware Requirements

The EduNexus platform is a web application that can be accessed via modern web browsers. As such, end-users (students or instructors) do not need specialized hardware beyond a device capable of running a web browser (e.g., a standard PC, laptop, or smartphone with internet access). However, for hosting the application and during development, the following hardware considerations were noted:

- **Server/Hosting:** A server machine with at least a dual-core processor, 4 GB of RAM, and sufficient storage (20 GB or more) to host the web server and database. These specifications would comfortably handle a moderate user base and can be scaled up for larger deployments.
- **Development Machines:** Each developer used a personal computer with at least 8 GB of RAM and a modern processor (Intel i5/AMD equivalent or better) to run the development environment, including running a local database and server for testing.
- **Internet Connectivity:** Reliable broadband internet is required on the server side (for cloud deployment or hosting) to serve content to users, and on the client side for users to interact with features like quiz generation and downloading resumes.

3.2 Software Requirements

The development and execution of EduNexus rely on a specific technology stack and software tools. Key software requirements include:

- **Operating System:** The development was primarily done on *Windows 10* and *Ubuntu 20.04 LTS*. The server can run on any OS that supports Node.js (Linux is preferred for production deployments).
- **Front-end Technologies:** *React.js* (version 18+), *HTML5*, *CSS3*, and *JavaScript (ES6+)*. *Tailwind CSS* is used for UI design and *Redux Toolkit* for state management.
- **Back-end Technologies:** *Node.js* (version 16+) with *Express.js* for the RESTful API server. *MongoDB* with *Mongoose* is used for the database. Authentication and security rely on *jsonwebtoken*, *bcrypt*, and *Nodemailer*.
- **AI Integration:** The *Gemini API* is used to dynamically generate quiz questions and cover letters based on user input.
- **Payment Gateway:** *Razorpay* integration is used for secure course enrollment and payment processing.
- **Development Tools:** *Visual Studio Code*, *Git* and *GitHub* for version control, and *Postman* for API testing. Deployment can be done using *Vercel* for the front-end and *Heroku* or *Railway* for the back-end.

3.3 Functional Requirements

Functional requirements describe the core features and capabilities that the EduNexus platform must provide. These were derived from the project objectives and refined through system analysis. The main functional requirements are:

- **User Registration and Authentication:** Users can register with name, email, and password, and activate their account via email OTP. JWT tokens will maintain login sessions securely. A password reset feature via email is included.
- **User Roles and Access Control:** EduNexus supports three roles—*Student*, *Instructor*, and *Admin*—each with specific access permissions and dashboards.
- **Course Creation and Management:** Instructors can create structured courses by adding metadata (title, category, description, price) and uploading video content organized into modules and lessons.

- **Course Enrollment and Consumption:** Students can browse available courses, filter by category or keyword, and enroll. Upon enrollment, they gain access to videos, progress tracking, and other course materials.
- **Payments and Transactions:** Students can pay for paid courses using Razorpay. The system generates and verifies orders before confirming enrollment and stores all transaction details securely.
- **Quiz Generation:** Students can enter a topic and number of questions to generate a quiz using the Gemini API. Questions and options are dynamically generated. After submitting, the student receives an instant result with scores and correct/incorrect answers. Instructors do not create quizzes manually.
- **Resume Builder:** Students can input academic, professional, and personal information into structured forms. The system dynamically creates and allows them to download a professional resume in PDF format using pre-defined templates.
- **Cover Letter Generator:** Students input job title, company name, and job description. The system uses the Gemini API to generate a personalized cover letter. Output is previewable and downloadable.
- **Profile Management:** Each user has a profile page to manage their information, courses, and tools (resume/cover letter/quiz history). Instructors manage their course lists, and Admins moderate content and users.
- **Rating and Review System:** After completing a course, students can provide star ratings and feedback. These are shown on the course page. Instructors and admins can respond to reviews.

3.4 Non-Functional Requirements

In addition to the above functional aspects, EduNexus must satisfy certain non-functional requirements which ensure the system's usability, reliability, and performance:

- **Usability:** The interface should be intuitive and easy to navigate. All features (course search, quiz, resume generation, etc.) must be accessible without a steep learning curve. The layout must be responsive across screen sizes and devices.
- **Performance:** The platform must be optimized for fast loading. Quiz generation using Gemini and resume creation must occur without significant delay. The back-end should efficiently process concurrent requests.

- **Scalability:** As the number of users and courses grows, the system should scale with minimal architectural changes. Horizontal scaling and modular architecture enable future extensions.
- **Security:** All sensitive data (passwords, emails, transactions) must be encrypted and securely stored. JWT and role-based access control must prevent unauthorized access. The platform should resist common vulnerabilities like XSS, CSRF, and injection attacks.
- **Reliability:** The system should ensure data consistency even in case of failure. Quiz results, resume data, and transactions must be saved persistently. Frequent backups should be configured.
- **Maintainability:** The codebase must follow clean, modular architecture. Logs should track system activity and errors for monitoring and debugging. Code should be well-documented for future contributors.
- **Extensibility:** The design should support easy integration of future features, such as job boards, AI-based recommendation engines, or additional quiz formats. REST APIs and reusable components support rapid enhancements.

Chapter 4

System Design

Designing the EduNexus platform involved creating a blueprint of how the system’s components interact and how information flows between them. This chapter covers the core design artifacts, including the use case analysis of the system, the structural design of the data model, and a representative workflow of the system’s operation.

4.1 Use Case Diagram

The use case diagram provides an overview of the interactions between different types of users (actors) and the major functionalities (use cases) of the EduNexus system. Figure 4.1 illustrates the primary use cases for each actor in the system:

- **Student:** A student can register, log in, browse courses, enroll in them (free or paid), and access structured content. They can generate quizzes by entering a topic and number of questions, view quiz results, and use the resume builder and cover letter generator tools. Students can also rate and review completed courses and manage their profile information.
- **Instructor:** In addition to student capabilities, instructors can create, edit, and publish courses. They manage content like videos and descriptions, structure lessons into modules, and monitor enrolled student lists. Instructors also maintain a detailed profile including contact and bio details.
- **Admin:** The admin oversees the platform’s operational health. Admins can manage users, approve or remove courses, moderate reviews, and access system settings. They ensure content and platform usage comply with standards and handle user escalation if needed.

The use case diagram visually separates shared and role-specific functionality. For example, generating quizzes, building resumes, and generating cover letters are available only to students, while course creation and publishing are restricted to instructors.

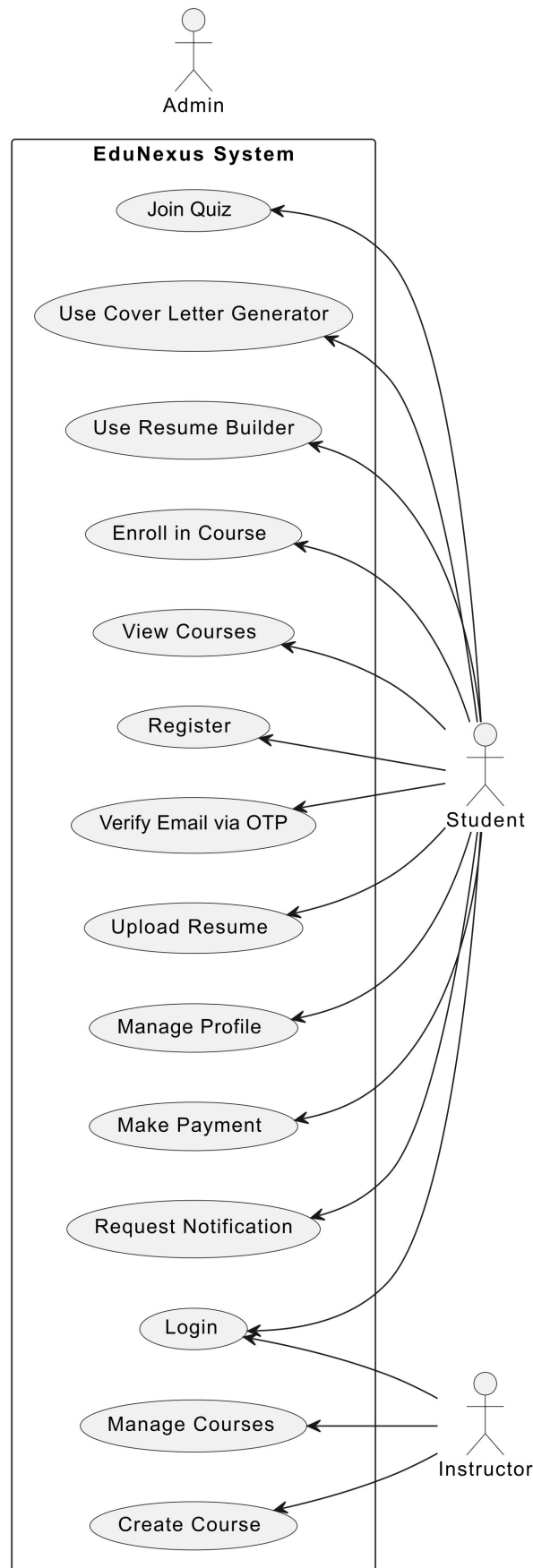


Figure 4.1: Use Case Diagram of the EduNexus system, depicting actors (Student, Instructor, Admin) and their interactions with the platform.

4.2 Class Diagram (Data Model)

The class diagram (or entity-relationship model) shows the static structure of EduNexus—its data entities, key attributes, and their relationships. Figure 4.2 outlines the key components:

- **User:** Stores core user credentials and role information (Student, Instructor, Admin). Linked to a Profile object and may contain references to enrolled courses, quiz attempts, generated resumes, and cover letters.
- **Course:** Represents a course created by an instructor. Includes metadata (title, description, price, status) and a list of Sections and Reviews. Stores enrolled student references and is tagged with categories.
- **Section & SubSection:** Each course consists of Sections, which contain multiple SubSections. SubSections hold video URLs, lesson descriptions, and durations.
- **RatingAndReview:** Captures student feedback on a course. Links to a User and Course, and stores rating, comments, and timestamps.
- **QuizSession:** Represents a quiz generated by the student. Includes topic, number of questions, and Gemini-generated questions. Also stores student’s selected answers and result summary.
- **Resume:** Represents a resume generated by the student. Stores resume content as structured fields (name, education, experience, etc.) and a generated PDF path or link.
- **CoverLetter:** Captures job-specific input and the AI-generated cover letter response. Includes job title, company name, job description, and generation timestamp.
- **Profile:** Stores additional personal information about a user (gender, bio, contact number, etc.) separate from login credentials for modularity.
- **OTP:** Stores email verification or password reset codes, with expiration and email mapping.

Relationships between entities:

- One **User** has one **Profile**.
- A **Course** has many **Sections**, each with many **SubSections**.
- A **Course** has many **RatingAndReview**.

- A **User** has many **QuizSession**, **Resume**, and **CoverLetter** entries.

This design supports scalability and extensibility, ensuring the system can accommodate evolving educational and career features.

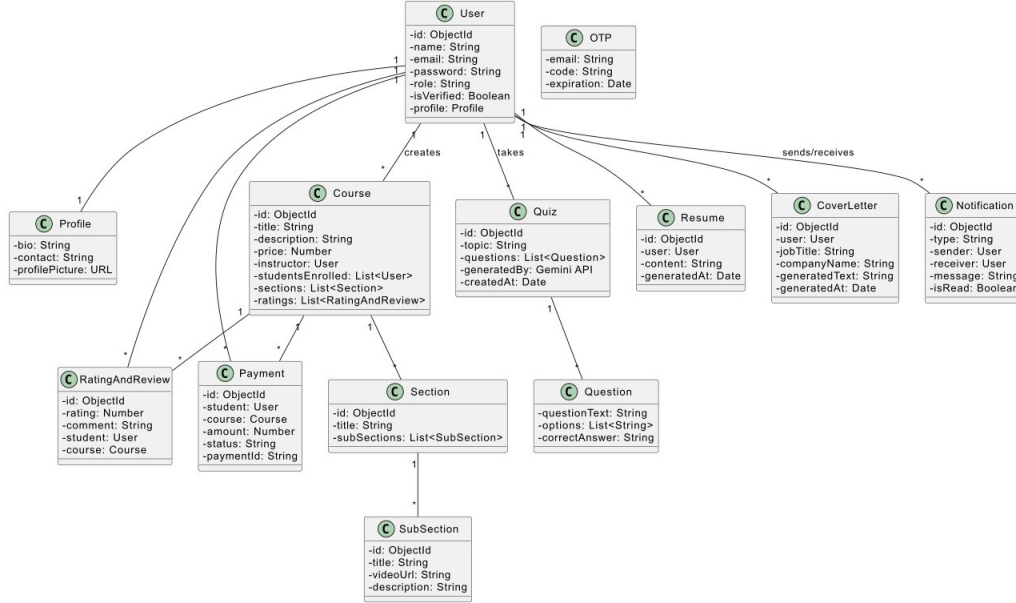


Figure 4.2: Class Diagram (ER Diagram) of the EduNexus system, showing the primary data models (User, Course, Quiz, Resume, etc.) and their relationships.

4.3 Activity Diagram: Course Enrollment Process

While the use case and class diagrams show static aspects of the system, the activity diagram focuses on a dynamic workflow. Figure 4.4 depicts the sequence of actions in the **Course Enrollment and Learning** process, from a student's perspective, highlighting interactions with the system:

1. **Browse Courses:** The student navigates the course catalog and selects a course to view its details (description, syllabus, price, reviews).
2. **Initiate Enrollment:** If the student decides to enroll, they click the enroll or purchase button. The system checks if the user is logged in and has not already enrolled in the course.
3. **Payment (if applicable):** For a paid course, the system initiates the payment gateway process (Razorpay). The student enters payment details and confirms the payment. The payment gateway processes the transaction and returns a success or failure status. In case of payment failure, the enrollment is aborted and the student may retry or cancel.

4. **Enrollment Confirmation:** Upon successful payment (or immediately for free courses), the system adds the student to the course’s enrolledStudents list and records a CourseProgress entry for that student (to track progress). A confirmation message or email is sent to the student. The course now appears in the student’s “My Courses” dashboard.
5. **Access Course Content:** The student can now access all sections and lessons of the course. They begin learning by clicking on a lesson. The system serves the video content and marks the lesson as in-progress or completed as the student watches it.
6. **Track Progress:** As the student goes through lessons, the system updates their progress (for example, marking lessons as completed). The student can see a progress indicator (e.g., 5/10 lessons completed).
7. **Completion and Feedback:** When the student finishes all course content, the course is considered completed. The student is prompted (optionally) to provide a rating and review for the course. They can also download a certificate if the platform issues one (not implemented in the initial scope but potentially a future enhancement).
8. **Continuous Engagement:** The activity diagram ends with course completion, but it can loop back if the student enrolls in another course. Moreover, the instructor of the course might host a Q&A webinar (bootcamp) as a follow-up, which the student could join, thus extending the interaction beyond the static content.

Throughout this process, various decision points (diamonds in the activity diagram) handle different scenarios: e.g., if the user is not logged in at the time of enrollment, they will be asked to log in first; if the payment fails, the system will not enroll the user; if the course is free, the payment step is skipped; etc. This activity flow ensures that the system logic for enrollment is clearly defined and that the user experience is smooth (with clear feedback at each step). By designing this flow beforehand, the implementation in code (especially managing states like “pending payment” or “completed”) becomes much more straightforward.

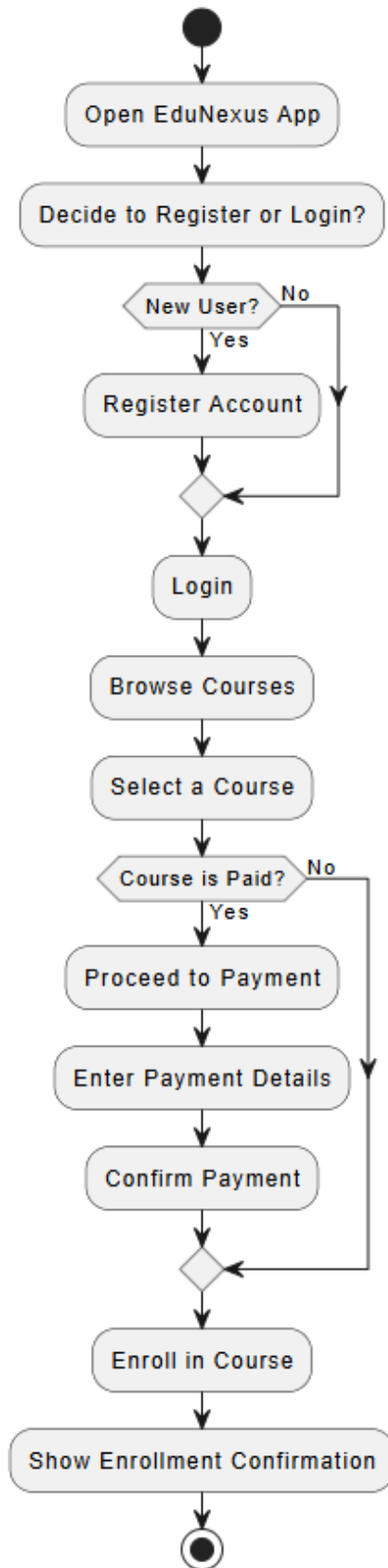


Figure 4.3: Activity Diagram illustrating the process of course enrollment and content access in EduNexus. It shows the student's journey from selecting a course, making payment, accessing lessons, to completing the course and giving feedback.

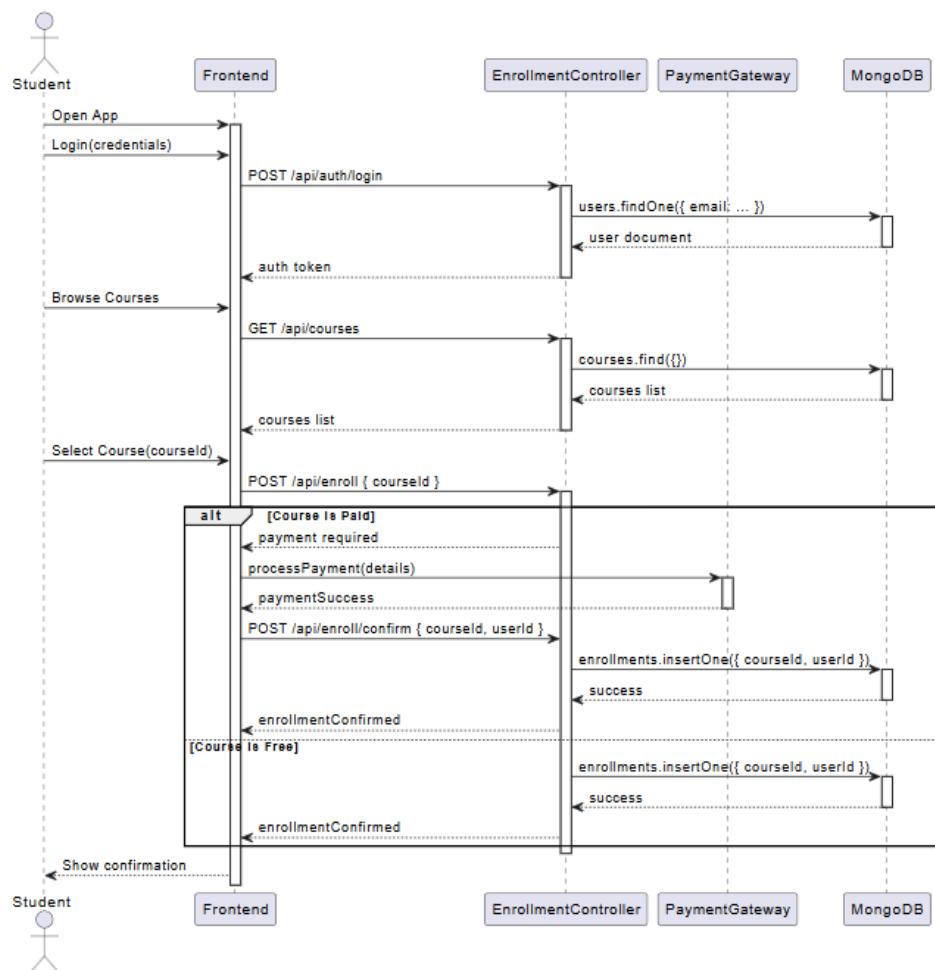


Figure 4.4: Sequence Diagram representing the flow of information.

Chapter 5

Implementation

The EduNexus platform was developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). This chapter presents how key functionalities were implemented, covering both front-end and back-end components. React was used for building responsive, interactive interfaces, while Express.js and Node.js powered RESTful API endpoints. MongoDB served as the NoSQL database.

5.1 Authentication and User Management

User authentication is JWT-based. During registration, React forms collect user information (name, email, password, role) and send it to the `/auth/register` endpoint. The backend hashes the password using Bcrypt and saves the user in MongoDB. An OTP is generated, stored temporarily, and sent to the user's email using NodeMailer for verification.

Login requests (`/auth/login`) are validated by comparing the hashed password. A JWT is generated and returned, which the client stores (usually in a secure cookie or local storage). Middleware on protected routes (e.g., `auth`, `isInstructor`) verifies user role and identity.

Profiles can be updated via `/profile`. Users may change profile info, bio, and avatar (stored in Cloudinary). Instructor resumes are also uploaded via the same system. Role-specific UI elements (like “Add Course” for instructors or “Dashboard” for admins) are conditionally rendered using role data from Redux state.

5.2 Course Module Implementation

Courses are created through a multi-step form in React. Instructors can add meta-data (title, description, price, etc.), sections, and lessons. Submitted data is sent to `/course/create`. The backend processes the request as follows:

1. Creates a Course document in MongoDB.
2. Iteratively creates Section and SubSection documents.
3. Stores uploaded videos using Cloudinary, or accepts external URLs.
4. Populates course content references (Section and SubSection IDs).

Courses are fetched via `/courses` and displayed on the catalog page. Details are loaded from `/course/:id` with nested population for curriculum display.

Course enrollment is handled through the `/course/enroll` route. Upon enrollment (manual or via Razorpay), the system updates the course's `studentsEnrolled` field and creates a `CourseProgress` record. React pages use Redux and protected routes to control access to course materials.

Course progress is tracked via `/course/update-progress`, and reviews can be submitted through `/course/:id/review`. The server validates review eligibility (e.g., course completion) and saves ratings.

5.3 Quiz Generator

Students can generate quizzes by submitting a topic and number of questions via a front-end form. A request is sent to `/quiz/generate`, where the server crafts a prompt and sends it to the Gemini API.

The response includes a list of questions with multiple-choice options and answers. These are rendered in the React UI as a quiz interface. After submission, results are calculated client-side or verified with the backend and displayed with score, correct/incorrect answers, and explanations.

This feature does not involve instructor-created quizzes—everything is dynamically generated using AI.

5.4 Resume Builder Implementation

The resume builder allows students to generate professional resumes by filling structured forms (e.g., education, experience, projects). Form data is submitted to `/resume/generate`.

The backend formats the data into a predefined HTML template and converts it into a PDF using libraries like `pdfkit` or `puppeteer`. The PDF is either directly downloadable or stored temporarily via Cloudinary.

This feature helps students quickly create industry-standard resumes without requiring design or layout knowledge.

5.5 Cover Letter Generator

Students can generate tailored cover letters by entering:

- Job title
- Company name
- Job description (optional)

This data is posted to `/coverletter/generate`. The server formats it into a Gemini API prompt and receives a customized letter. The result is shown in the React UI and can be copied or downloaded.

5.6 Payment Integration

Razorpay was used for secure payment handling. When a student purchases a paid course, the client calls `/payment/capture`, and the backend creates a Razorpay order using its Node SDK. The order ID is returned to the client, which invokes the Razorpay checkout popup.

After payment, Razorpay returns the payment ID and signature to the front-end. A verification request is sent to `/payment/verify`, and if validated, the student is enrolled in the course. Confirmation emails are sent using Nodemailer.

Duplicate purchases are prevented through server-side checks, and course access is immediately updated after successful payment.

5.7 Other Features and Admin Tools

- **Contact Form:** User messages submitted via `/contact` are emailed to platform admins using NodeMailer.
- **Search and Filtering:** Course catalog supports keyword and category-based filtering. Basic search is implemented client-side.
- **Admin Controls:** Admins can delete users, update roles, and manage courses using dedicated API endpoints.
- **Responsive Design:** Tailwind CSS and React components ensure the platform is fully responsive across devices. Reusable components like modals, buttons, and forms improve consistency and UX.

Chapter 6

Testing

Testing was conducted at multiple stages of the project to ensure the correctness and reliability of EduNexus. A combination of unit, integration, and user acceptance testing was performed. This chapter outlines the testing methodology, selected test cases, and results.

6.1 Test Plan and Methodology

We followed an iterative and manual testing approach. Each module was tested in isolation using tools like Postman (for API testing) and browser consoles (for front-end interactions). Once stable, integration tests were performed by simulating workflows across different user roles (Student, Instructor, Admin).

Key testing techniques included:

- **API testing:** Verifying HTTP response codes, request payloads, authentication headers, and error handling.
- **UI testing:** Manually exercising forms, validating input fields, checking conditional rendering, and cross-browser behavior.
- **Integration testing:** Running the full MERN stack locally and simulating real workflows such as course enrollment, payment, and quiz generation.
- **Security checks:** Attempting unauthorized access to protected endpoints using invalid tokens or mismatched roles.

6.2 Test Cases and Results

Here are representative test cases, grouped by major modules:

1. User Registration and Login

Test: Register with valid and invalid emails; login with correct and incorrect passwords.

Result: OTP email was received, invalid attempts were blocked, and successful login returned a JWT token.

2. Course Creation and Listing

Test: Instructor creates a course with multiple sections and lessons. **Result:** Course structure was saved properly. UI rendered the curriculum. Course appeared in the public catalog after publishing.

3. Course Enrollment and Payment

Test: Enroll in a paid course using Razorpay test credentials. **Result:** Order created and verified successfully. Enrollment recorded. Confirmation email sent. Duplicate enrollment attempts were blocked.

4. Quiz Generator

Test: Student enters a topic and number of questions to generate a quiz. **Result:** Gemini API returned structured questions and options. Quiz was rendered on the UI. After submission, score and correct answers were displayed.

5. Resume Builder

Test: Fill out the resume form with various inputs (education, experience, skills). **Result:** Resume was generated and downloadable as PDF. Layout was clean and consistent across inputs.

6. Cover Letter Generator

Test: Submit job title, company name, and description. **Result:** Gemini API returned a coherent, professional letter. It was displayed with an option to download.

7. Forgot Password Workflow

Test: Trigger password reset and set a new password via OTP link. **Result:** Reset link received. New password saved and verified via login.

8. Unauthorized Access Prevention

Test: Access instructor-only endpoints using a student's token. **Result:** 403 Forbidden error returned. Role-based access control worked correctly.

9. Data Validation

Test: Submit forms with missing or invalid data (e.g., empty title, too-long bio, invalid video URL). **Result:** Front-end and back-end both returned validation errors. Forms displayed appropriate error messages.

10. Performance (Light Load Test)

Test: Simultaneously access course content from multiple student accounts. **Result:** The server handled concurrent requests without failure or slowdown. API responses remained under 1 second.

6.3 Summary

All core functionalities were tested successfully. Identified bugs (such as duplicate review submissions and large video upload failures) were resolved. While formal test automation (e.g., Jest, Cypress) was not implemented due to scope and time constraints, the current system is functionally complete and stable for real-world usage.

Future enhancements will include automated test coverage for APIs and components, as well as real-time performance monitoring.

Chapter 7

Results

7.1 Home

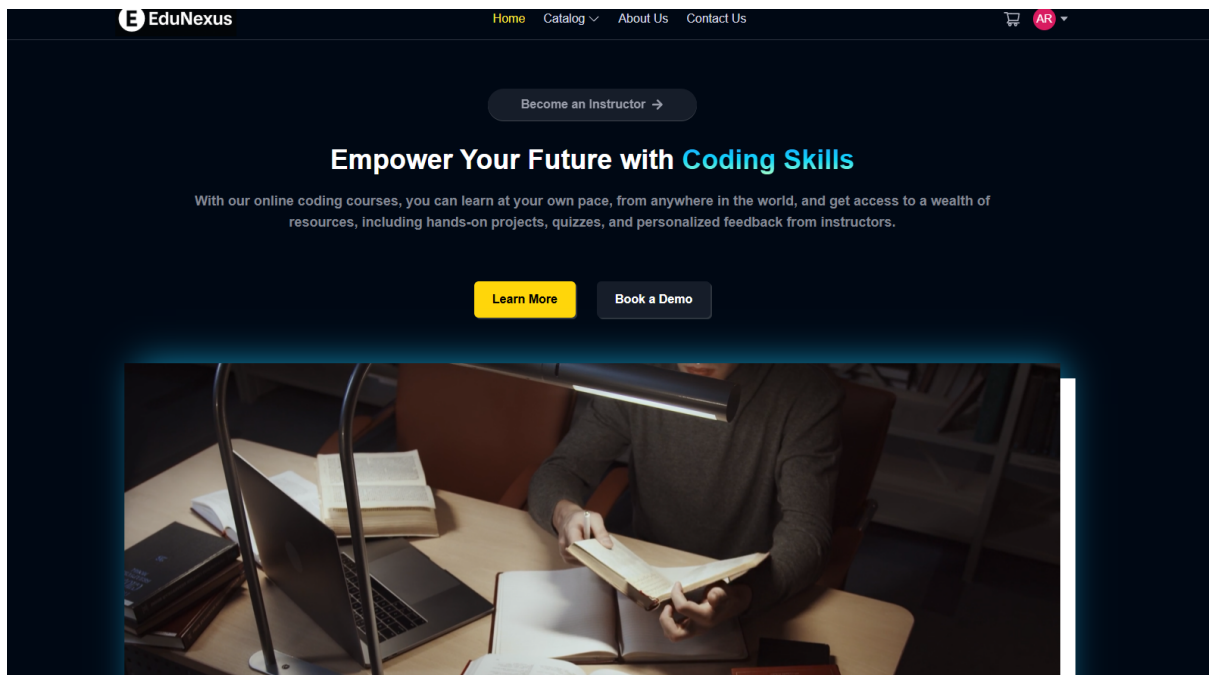
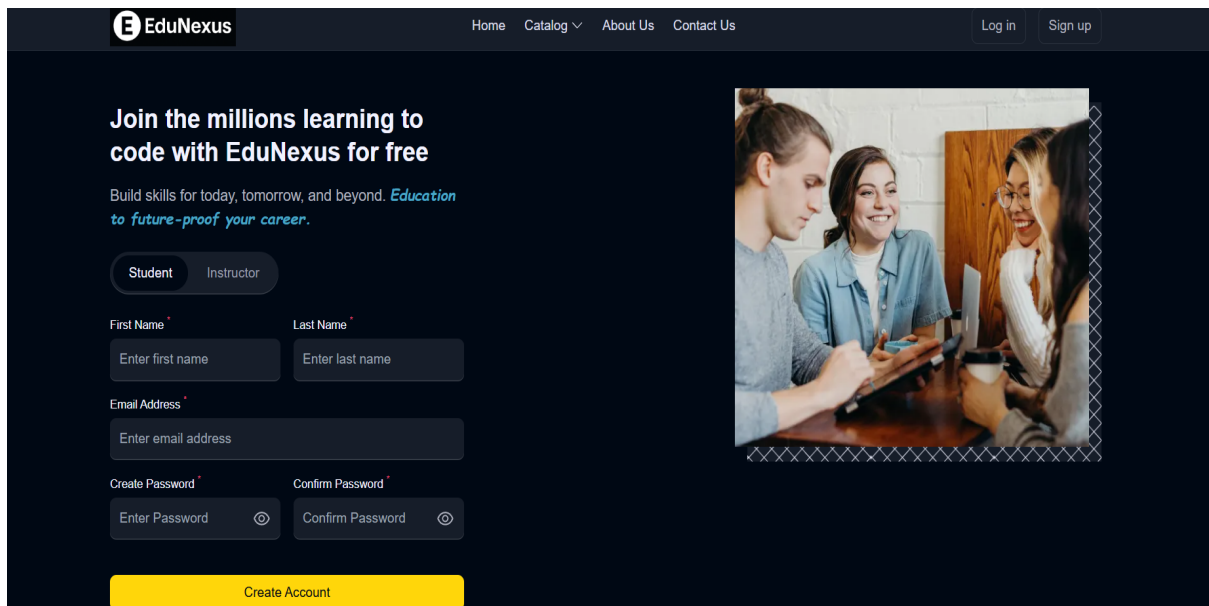


Figure 7.1: Home Page

7.2 Login and SignUp Page



The image shows the EduNexus login and sign-up page. The header includes the EduNexus logo, navigation links (Home, Catalog, About Us, Contact Us), and user links (Log in, Sign up). The main content area features a sign-up form with the heading "Join the millions learning to code with EduNexus for free" and a subheading "Build skills for today, tomorrow, and beyond. *Education to future-proof your career.*". The form has two tabs: "Student" (selected) and "Instructor". The fields include First Name, Last Name, Email Address, Create Password, and Confirm Password. A yellow "Create Account" button is at the bottom. To the right is an image of three people collaborating at a desk.

EduNexus Home Catalog About Us Contact Us Log in Sign up

Join the millions learning to code with EduNexus for free

Build skills for today, tomorrow, and beyond. *Education to future-proof your career.*

Student Instructor

First Name * Last Name *

Enter first name Enter last name

Email Address *

Enter email address

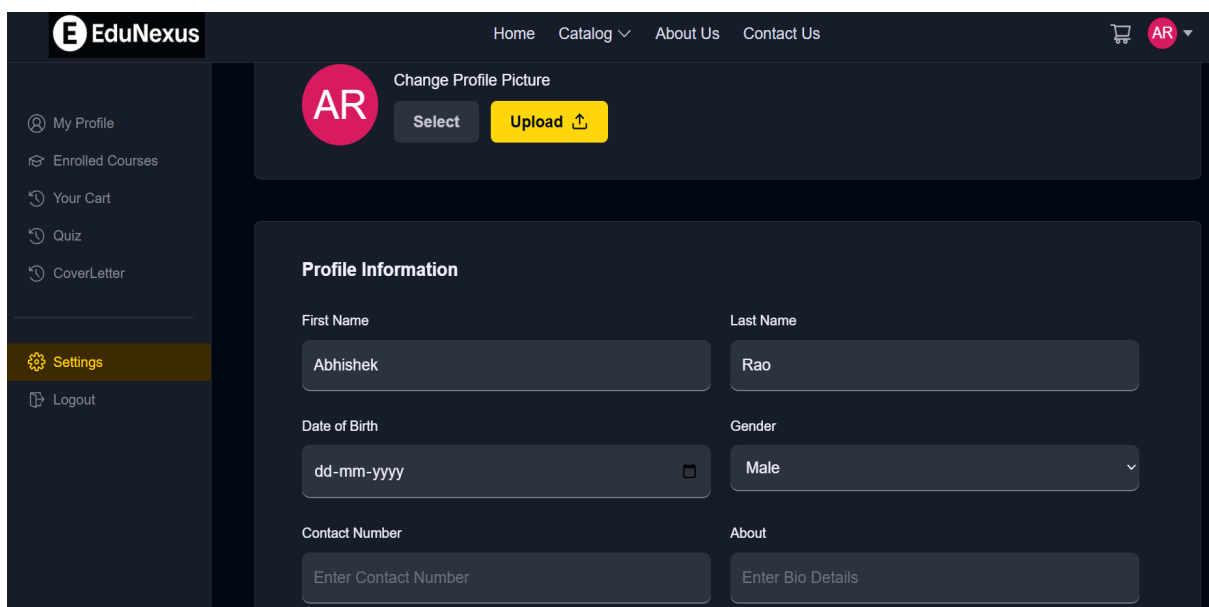
Create Password * Confirm Password *

Enter Password Enter Password

Create Account

Figure 7.2: Login and signup

7.3 Managing User Profiles



The image shows the EduNexus user profile management page. The header includes the EduNexus logo, navigation links (Home, Catalog, About Us, Contact Us), and a user profile icon (AR). The left sidebar contains links: My Profile, Enrolled Courses, Your Cart, Quiz, CoverLetter, Settings (highlighted), and Logout. The main content area has a "Change Profile Picture" section with a "Select" button and an "Upload" button. Below is the "Profile Information" section with fields for First Name (Abhishek), Last Name (Rao), Date of Birth (dd-mm-yyyy), Gender (Male), Contact Number, and About (Enter Bio Details).

EduNexus Home Catalog About Us Contact Us AR

Change Profile Picture

Select Upload

Profile Information

First Name Last Name

Abhishek Rao

Date of Birth Gender

dd-mm-yyyy Male

Contact Number About

Enter Contact Number Enter Bio Details

Figure 7.3: Managing user profiles.

7.4 Add to cart and Enrolled course

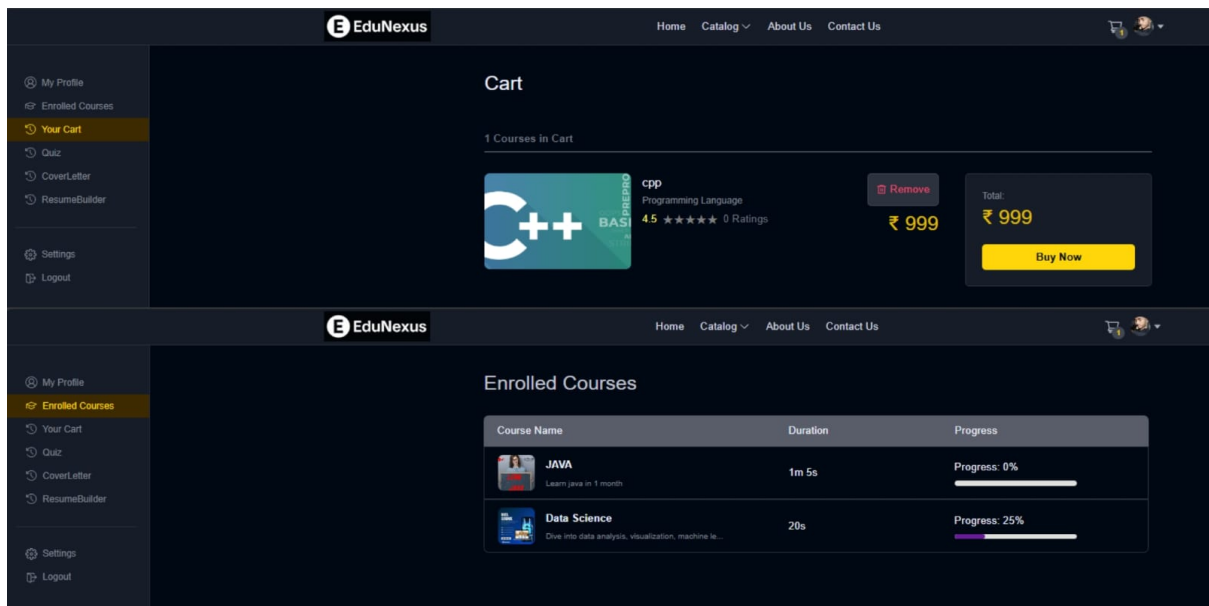


Figure 7.4: Cart and enrolled course.

7.5 Payment

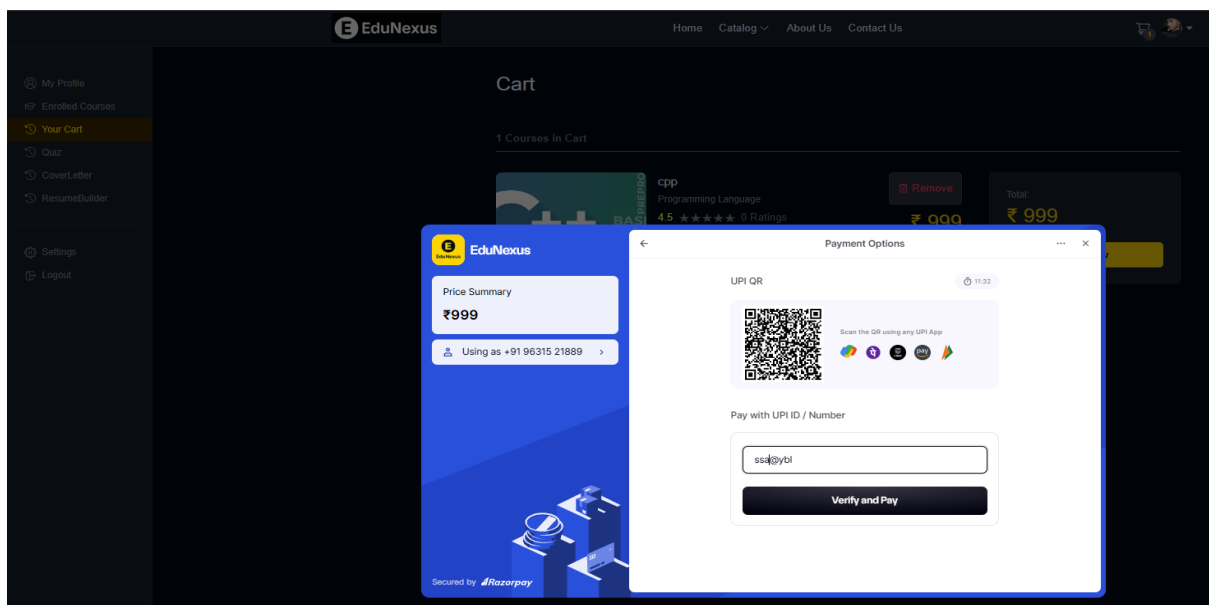


Figure 7.5: payment

7.6 Quiz page

EduNexus Home Catalog ▾ About Us Contact Us

My Profile
Enrolled Courses
Your Cart
Quiz
CoverLetter
ResumeBuilder
Settings
Logout

Personalized Learning Quiz System

Topic
e.g. Space, Python

Number of Questions
0

Difficulty
Easy ▾

Start Quiz

View Quiz Performance

Figure 7.6: quiz-page

7.7 Quiz Result

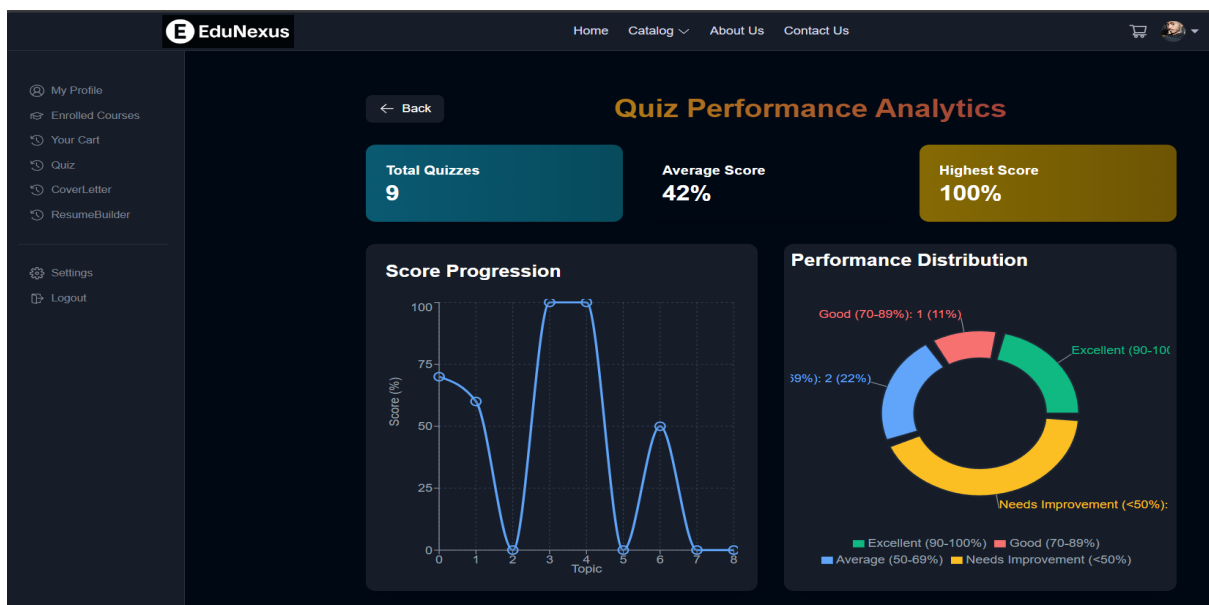


Figure 7.7: Quiz Dashboard

7.8 Resume Builder

The screenshot shows the 'Resume Builder' page on the EduNexus website. The left sidebar contains navigation links: My Profile, Enrolled Courses, Your Cart, Quiz, CoverLetter, ResumeBuilder (highlighted), Settings, and Logout. The main content area is titled 'Resume Builder' and features a 'Fill Your Resume Details' button. Below this, there are input fields for Profile Photo (with a 'Choose File' button and 'No file chosen' text), Full Name, Email, Phone, Address, LinkedIn URL, and GitHub URL. A large text area is provided for the Professional Summary. At the bottom, there is a 'Skills' section with an 'Enter a skill' input field and an 'Add' button. On the right side, a vertical list of resume sections is shown: Professional Summary, Skills, Achievements, Certifications, Languages, Education, Experience, and Projects. Each section has a corresponding icon and a link to its respective form.

Figure 7.8: Resume Builder

7.9 Cover Letter Generator

The screenshot shows the 'JobPrep Writer' page on the EduNexus website. The left sidebar contains navigation links: My Profile, Enrolled Courses, Your Cart, Quiz, CoverLetter (highlighted), ResumeBuilder, Settings, and Logout. The main content area is titled 'JobPrep Writer' and features input fields for Company Name (with a placeholder 'e.g., Google'), Job Title (with a placeholder 'e.g., Frontend Developer'), and Job Description (with a placeholder 'Paste the job description here...'). A 'Generate Cover Letter' button is located at the bottom right of the form.

Figure 7.9: Cover Letter

7.10 Available Categories of Course

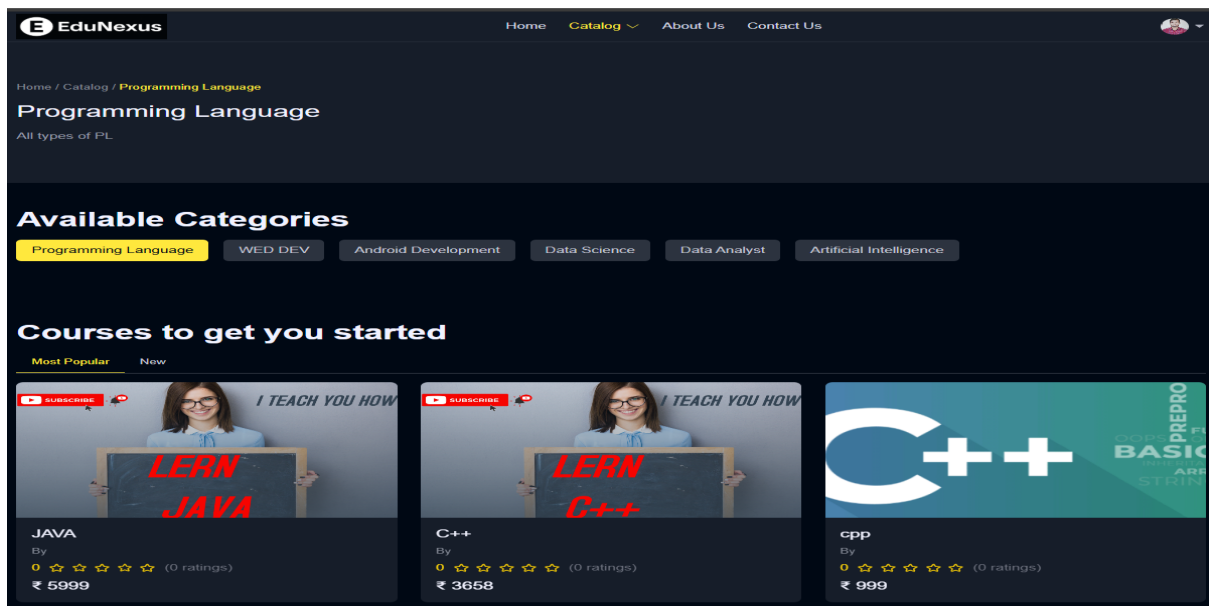


Figure 7.10: Available-Categories

7.11 Description of Course detail

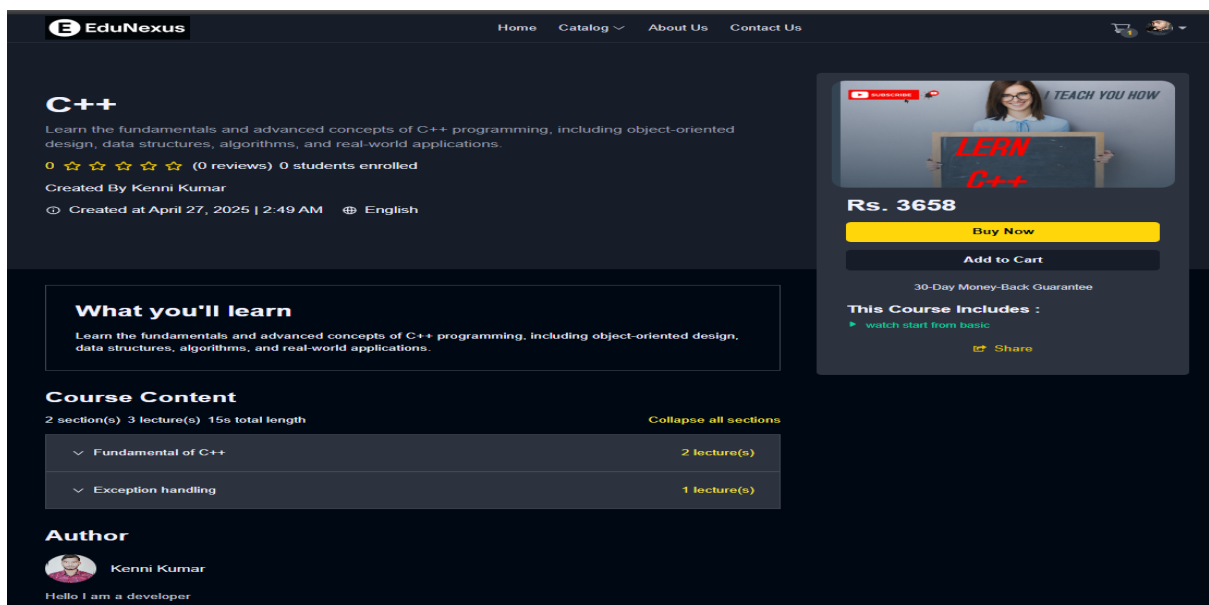


Figure 7.11: Course-detail

7.12 Watching the video of the Course

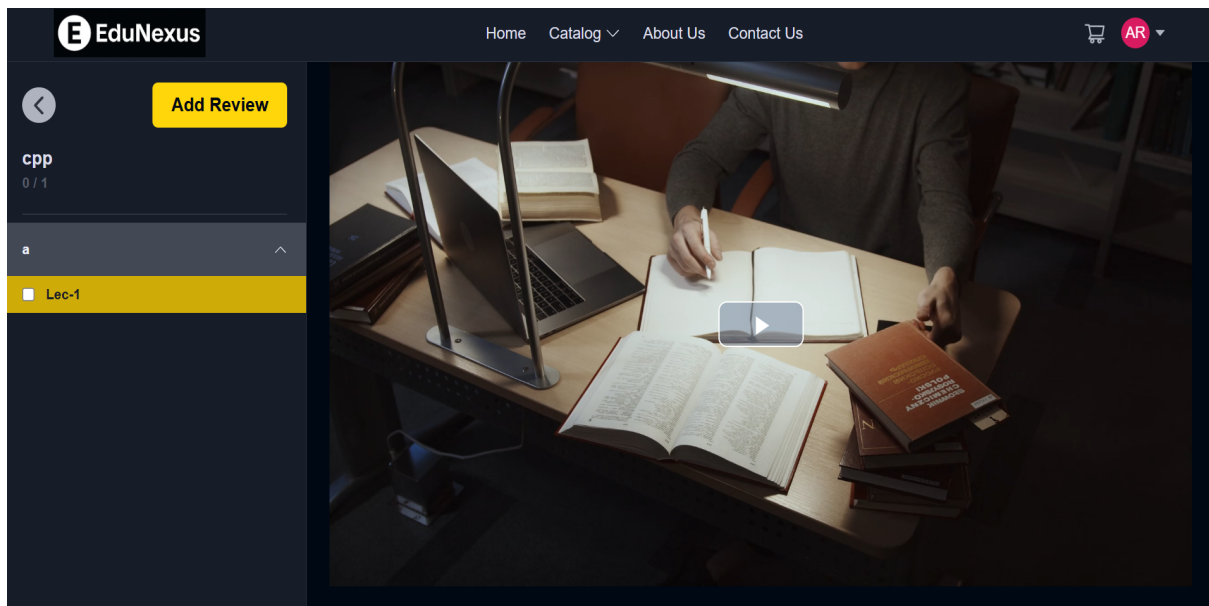


Figure 7.12: Watching the video

7.13 Rating and Feedback

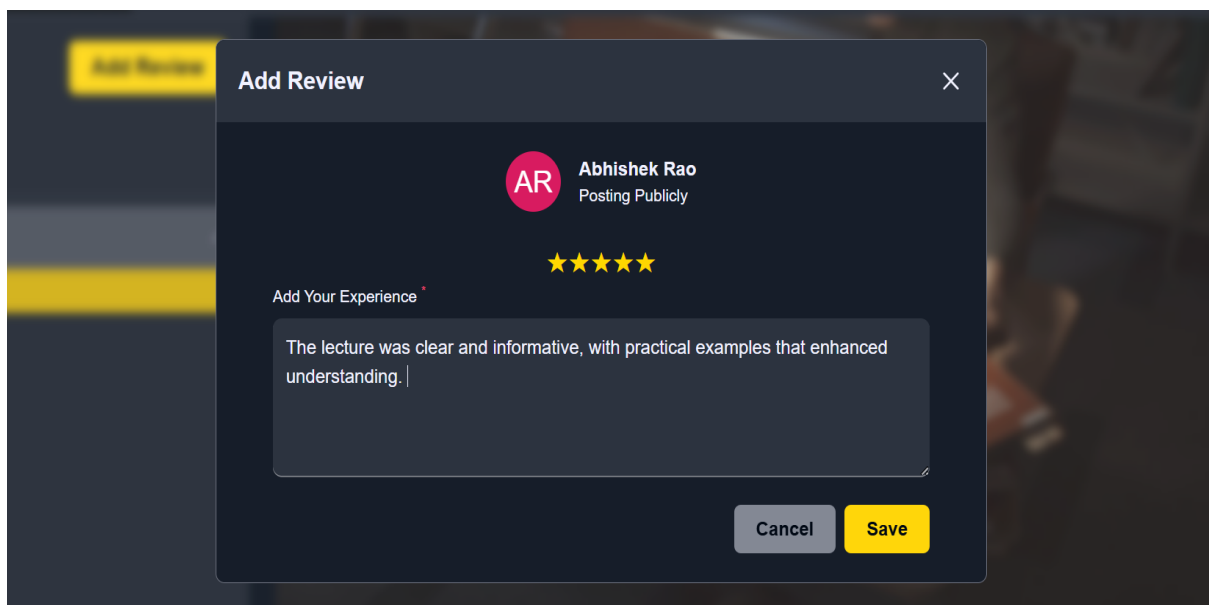
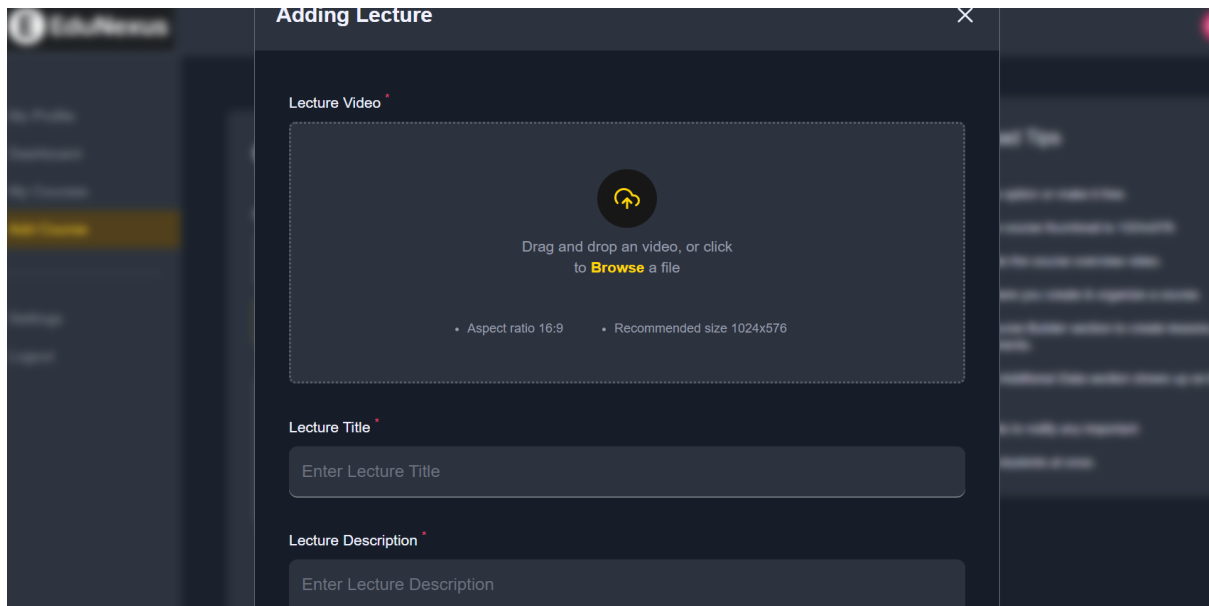


Figure 7.13: Rating and Feedback

7.14 Adding Lecture By Instructor

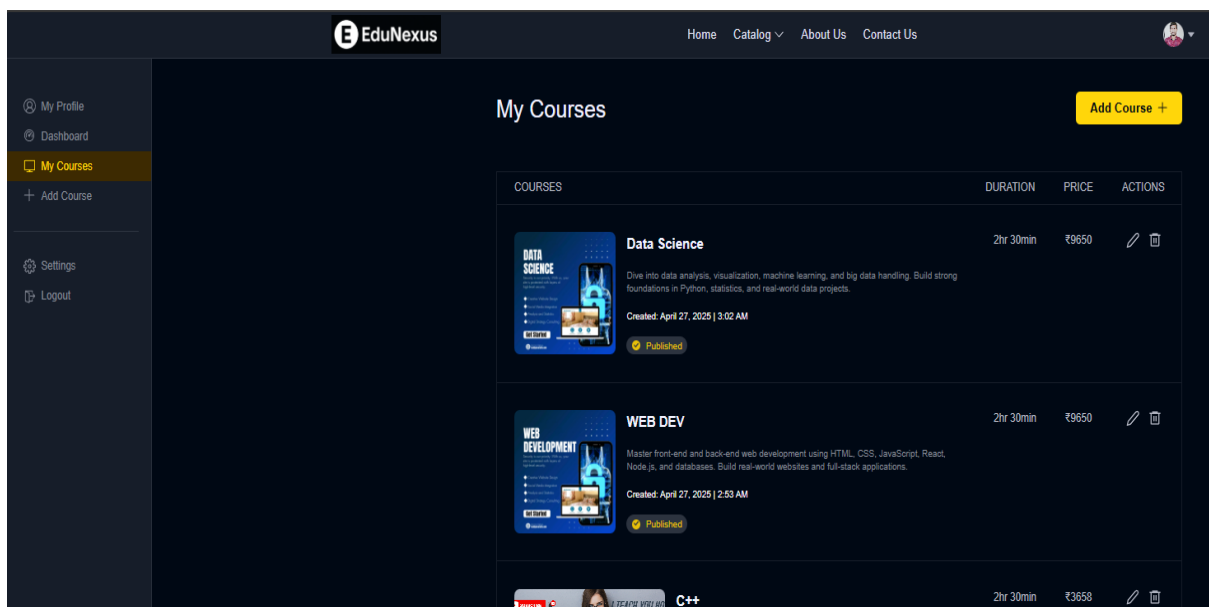


The screenshot shows a modal window titled "Adding Lecture" with a close button (X) in the top right corner. The form contains the following fields:

- Lecture Video ***: A large dashed box for video upload. Inside, there is a circular icon with a video camera and the text "Drag and drop an video, or click to **Browse** a file". Below this, two requirements are listed: "Aspect ratio 16:9" and "Recommended size 1024x576".
- Lecture Title ***: A text input field with the placeholder text "Enter Lecture Title".
- Lecture Description ***: A text input field with the placeholder text "Enter Lecture Description".

Figure 7.14: New lec..

7.15 ALL Course of Instructor



The screenshot shows the "My Courses" page in the EduNexus application. The page has a dark theme and a sidebar on the left with navigation links: "My Profile", "Dashboard", "My Courses" (highlighted), "+ Add Course", "Settings", and "Logout". The main content area displays a table of courses with the following columns: "COURSES", "DURATION", "PRICE", and "ACTIONS".

COURSES	DURATION	PRICE	ACTIONS
 Data Science Dive into data analysis, visualization, machine learning, and big data handling. Build strong foundations in Python, statistics, and real-world data projects. Created: April 27, 2025 3:02 AM Published	2hr 30min	₹9650	 
 WEB DEV Master front-end and back-end web development using HTML, CSS, JavaScript, React, Node.js, and databases. Build real-world websites and full-stack applications. Created: April 27, 2025 2:53 AM Published	2hr 30min	₹9650	 
 C++	2hr 30min	₹3658	 

Figure 7.15: All course of instructor

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project culminated in the development of **EduNexus**, a modern e-learning platform that integrates self-paced courses and career support tools into a single web application. It addresses key challenges in digital education, such as limited interactivity, fragmented tools, and lack of integrated job preparation support.

EduNexus implements role-based functionality for Students and Instructors. Students can register, explore courses, securely enroll via Razorpay, and access structured video content. Instructors are empowered to create and manage course modules through an intuitive interface. The platform incorporates features like a resume builder and AI-powered cover letter generator, enabling users to enhance their career readiness directly within the learning system.

Built on the MERN stack (MongoDB, Express.js, React.js, Node.js), the platform benefits from a unified codebase and efficient data handling through JSON APIs. Secure authentication via JWT and password hashing ensures data safety. Third-party integrations, including Cloudinary for media storage and Gemini API for quiz generation, enabled rapid implementation of dynamic and intelligent features.

The system was tested manually across major workflows, such as registration, login, course creation, and quiz generation. The application performed reliably under standard scenarios, confirming its core stability and usability.

8.2 Future Work

While the current version of EduNexus successfully delivers its foundational features, several enhancements are planned to evolve the platform into a more comprehensive, engaging, and scalable learning ecosystem. Key areas of future development include:

1. Mobile and Cross-Platform Accessibility

To expand usability across devices, a mobile application or Progressive Web App (PWA) will be developed, allowing users to access course content and features conveniently from smartphones and tablets.

2. Live Sessions and Bootcamps

Future versions will introduce live class functionality and instructor-led bootcamps. These real-time interactive sessions will help bridge the gap between static learning and classroom-like engagement.

3. Assessment and Certification

Instructor-assigned tasks, submission-based assessments, and auto-evaluation quizzes will be added. Digital certificates will be generated upon course or bootcamp completion to validate learners' achievements.

4. Instructor Analytics Dashboard

A powerful analytics dashboard will provide instructors with insights into student participation, course progress, and quiz performance to help improve teaching strategies and learner outcomes.

5. Community Interaction via Forums

To support collaborative learning, a built-in discussion forum will allow students and instructors to engage in Q&A threads, discussions, and peer support directly on the platform.

6. Gamified Learning Experience

The platform will incorporate gamification features such as points, badges, levels, and leaderboards to enhance learner motivation, progress tracking, and course completion rates.

Bibliography

- [1] React – A JavaScript library for building user interfaces.
<https://reactjs.org/docs>
- [2] Redux – A Predictable State Container for JS Apps.
<https://redux.js.org/introduction/getting-started>
- [3] MDN Web Docs – HTML, CSS, and JavaScript reference.
<https://developer.mozilla.org/>
- [4] MongoDB Manual – MongoDB Documentation.
<https://docs.mongodb.com/manual/>
- [5] Express – Fast, unopinionated, minimalist web framework for Node.js.
<https://expressjs.com/en/guide/>
- [6] Mongoose – MongoDB object modeling for Node.js.
<https://mongoosejs.com/docs/guide.html>
- [7] Tailwind CSS – A utility-first CSS framework.
<https://tailwindcss.com/docs>
- [8] B. Hollister, P. Nair, S. Hill-Lindsay, and L. Chukoskie (2022). *Engagement in Online Learning: Student Attitudes and Behavior During COVID-19*. Frontiers in Education, 7:851019.
- [9] S. DeFlitch (2023). *What is Edtech? Key Benefits of Educational Technology in 2025*. Panorama Education Blog. [Online]. Available: <https://www.panoramaed.com/blog/what-is-edtech-key-benefits-of-educational-technology-in-2025>
- [10] Razorpay. (n.d.). *Razorpay Payment Gateway API Documentation*. [Online]. Available: <https://razorpay.com/docs/api/>